

Complete DevOps Interview Preparation Guide

Nagur Meera Narendra Gella · DevOps Engineer · 3.5 Years Experience TCS | Client: BT Group · Hyderabad, India
Tools: AWS · Terraform · Kubernetes · Docker · Jenkins · ArgoCD · Ansible AWX Maven · Nexus · SonarQube · Prometheus · Grafana · Dynatrace · SSL/TLS · Git · Linux

Section 1	Self Introduction	Short (30–45s) + Detailed (1–2 min) versions
Section 2	Project Explanation	High-level + step-by-step flow + challenges + solutions
Section 3	AWS	EC2, VPC, ALB, ASG, EKS, RDS, IAM, S3, CloudWatch, Route 53
Section 4	Terraform	Modules, remote state, state locking, workspaces
Section 5	Kubernetes	kubeadm, EKS, PKI, RBAC, node lifecycle, upgrades
Section 6	Docker	Images, containers, Dockerfile, networking, volumes
Section 7	Jenkins	Pipelines, Jenkinsfile, stages, plugins, integration
Section 8	ArgoCD & GitOps	Sync, drift, rollback, app management
Section 9	Helm	Charts, values, release management
Section 10	Ansible AWX	Playbooks, inventory, CyberArk integration
Section 11	Linux	Commands, processes, networking, troubleshooting
Section 12	Git (GitHub/GitLab)	Branching, merge, rebase, webhooks
Section 13	Maven	Build lifecycle, POM, dependency management
Section 14	Nexus	Repository management, artifact storage
Section 15	SonarQube	Code quality, quality gates, integration
Section 16	Prometheus & Grafana	Metrics, alerting, dashboards
Section 17	Dynatrace	APM, alerting, Kubernetes monitoring
Section 18	SSL/TLS	Certificates, Keytool, expiry, renewal

SECTION 1: Self Introduction

Short Version (30–45 seconds)

"I am Narendra, a DevOps Engineer with 3.5 years of experience at Tata Consultancy Services, working for BT Group as the end client.

My core work involves managing production Kubernetes clusters supporting 20+ microservices, implementing GitOps-based deployments using ArgoCD, and building CI/CD pipelines with Jenkins.

On the cloud side, I have hands-on experience with AWS — provisioning infrastructure using Terraform, working with EKS, ALB, RDS, and CloudWatch.

I have delivered measurable results: 60% reduction in deployment effort, 40% improvement in build consistency, and 30% faster incident resolution.

I hold the AWS Certified Cloud Practitioner certification and I am looking for a role where I can continue growing in cloud-native DevOps engineering."

Detailed Version (1–2 minutes)

"I am Narendra, a DevOps Engineer with 3.5 years of hands-on experience at TCS, working directly for BT Group — one of the UK's largest telecommunications companies.

My primary strength is Kubernetes. I manage self-managed kubeadm clusters supporting 20+ microservices across 4 environments — Dev, Staging, Production, and DR — maintaining 99.9% uptime SLA.

On the CI/CD side, I own the full pipeline: developers push code to GitLab, Jenkins picks it up, Maven builds and tests it, SonarQube quality-gates it, the artifact goes to Nexus, a Docker image is built and pushed to Harbor, and ArgoCD deploys it to Kubernetes via GitOps — zero manual intervention.

For AWS, I provisioned and managed cloud infrastructure using Terraform with reusable modules and S3/DynamoDB remote state — covering EKS, VPC with multi-AZ design, ALB, RDS MySQL Multi-AZ, EFS, ECR, CloudWatch, and IAM with least-privilege.

I also handle the operational side: certificate lifecycle management for 110+ servers, OS patching via Change Request windows, PCI DSS compliance evidence, and runbook-driven incident resolution using Dynatrace and Prometheus/Grafana.

Key results: ~60% reduction in deployment effort through Ansible AWX automation, ~40% improvement in build consistency through standardized CI/CD, ~30% faster incident resolution through structured RCA and runbooks, and ~15% AWS cost reduction through right-sizing and lifecycle policies.

I hold the AWS Certified Cloud Practitioner certification. I am looking for a DevOps role where I can deepen my cloud-native skills and take more ownership of infrastructure design."

SECTION 2: Project Explanation

2.1 High-Level Explanation (Simple Version)

At BT Group, I worked on two projects: a live production Kubernetes platform (on-premises) and an AWS cloud migration project.

On-Prem Project: A self-managed Kubernetes platform running Java/Spring Boot microservices and Tomcat web apps across 4 environments. Traffic flows through VTM (prod) / HAProxy (non-prod) → Kubernetes Services → Pods. Data stored in Oracle/MySQL. Shared storage via NFS as PersistentVolumes. CI/CD via Jenkins + Maven + SonarQube + Nexus + Docker + Harbor + ArgoCD.

AWS Project: Provisioned equivalent cloud infrastructure using Terraform. Route 53 → ALB → EKS → Pods → RDS MySQL Multi-AZ + EFS. Everything automated as code — reusable Terraform modules, S3/DynamoDB remote state.

2.2 Step-by-Step CI/CD Flow

Step 1 — Developer commits code

Developer pushes code to GitLab (on-prem) / GitHub (AWS). Raises Merge Request. After review and approval, code merges to main branch. GitLab/GitHub webhook triggers Jenkins automatically.

Step 2 — Jenkins pipeline starts

Jenkins (standalone VM) picks up the webhook. Checks out source code. Runs the Jenkinsfile pipeline — a declarative pipeline as code committed in the repo itself.

Step 3 — Maven build

mvn clean package runs. Compiles Java code, executes unit tests, packages WAR (Tomcat) or JAR (Spring Boot). If tests fail — pipeline stops. No artifact created.

Step 4 — SonarQube scan

Jenkins triggers SonarQube analysis. Scans for bugs, vulnerabilities, code smells, coverage. Quality Gate checks results. If gate fails (e.g., coverage < 80% or critical CVE) — pipeline stops. This enforced quality and gave us ~40% better build consistency.

Step 5 — Nexus artifact storage

Approved WAR/JAR pushed to Nexus with version tag. Nexus is the artifact repository — stores every version for traceability and rollback. In AWS, artifacts go to S3 instead.

Step 6 — Docker image build

Jenkins runs docker build. Dockerfile pulls artifact from Nexus, packages it into a Docker image tagged with version + git commit hash.

Step 7 — Push to registry

On-prem: docker push to Harbor registry. Harbor scans for CVEs automatically. AWS: docker push to Amazon ECR. ECR also has built-in vulnerability scanning.

Step 8 — ArgoCD GitOps trigger

Jenkins updates the image tag in the Kubernetes manifest Git repo (separate repo from app code). ArgoCD continuously watches this manifest repo. Detects the new commit. Compares desired state (Git) vs live state (cluster). Finds diff. Syncs cluster to match Git.

Step 9 — Kubernetes rolling update

Kubernetes performs rolling update — starts new pod with new image, waits for readiness probe, terminates old pod. Zero downtime. If new pod fails readiness, rollout pauses — old pods keep serving traffic.

Step 10 — Post-deployment monitoring

Dynatrace detects deployment event, compares pre/post metrics. Prometheus/Grafana dashboards show error rate, pod restarts, response time. CloudWatch (AWS) monitors ALB 5xx and EKS node metrics. Any spike triggers alert via Prometheus Alertmanager or CloudWatch + SNS.

2.3 Your Role and Responsibilities

- Owned and managed production Kubernetes clusters (kubeadm) — cluster upgrades, certificate lifecycle (PKI rotation, kubeadm certs renew), node patching (cordon/drain/uncordon)

- Managed CI/CD pipelines — Jenkins pipeline creation, SonarQube quality gate configuration, Nexus artifact management, Docker image builds
- Implemented GitOps using ArgoCD — configured applications, managed sync policies, resolved OutOfSync/drift states
- Provisioned AWS infrastructure using Terraform — VPC, EKS, ALB, RDS, EFS, ECR, IAM with reusable modules and remote state
- Automated Tomcat deployments via Ansible AWX integrated with CyberArk — achieved 60% deployment effort reduction
- Managed SSL/TLS certificate lifecycle on 110+ servers using Java Keytool — zero unplanned expiry incidents
- Built and maintained monitoring — Dynatrace APM, Prometheus/Grafana metrics, CloudWatch dashboards and alarms
- Maintained PCI DSS compliance — access reports, change logs, vulnerability dashboards — zero major audit findings
- OS patching of 110+ Linux servers within Change Request windows via ServiceNow

2.4 Real Challenges and How You Solved Them

Challenge 1: Kubernetes Certificate Expiry

Problem: Control plane API server became unreachable — all kubectl commands failing with TLS errors.

Steps taken:

- Used crictl on the control plane node to check container status (kubectl not working).
- Ran: kubeadm certs check-expiration — confirmed multiple certs expired.
- Ran: kubeadm certs renew all — renewed all control plane certificates.
- Restarted static pod manifests under /etc/kubernetes/manifests.
- Updated kubeconfig: copied new admin.conf for kubectl access.
- Verified: kubectl get nodes returned successfully.
- Added to runbook + set up proactive Dynatrace alerting for cert expiry 30 days in advance.

Challenge 2: Pod CrashLoopBackOff in Production

Problem: Multiple pods going into CrashLoopBackOff — application team reporting service unavailable.

Steps taken:

- kubectl get pods -n prod — confirmed CrashLoopBackOff status.
- kubectl logs --previous — found OOMKilled error (memory limit exceeded).
- kubectl describe pod — confirmed memory limit set too low for the workload.
- Short-term fix: increased memory limit in Deployment manifest, ArgoCD synced the change.
- Long-term fix: worked with dev team to profile app memory usage, set appropriate limits.
- Added Prometheus alert for OOMKilled events to catch early.

Challenge 3: Jenkins Pipeline Failure — SonarQube Quality Gate

Problem: Jenkins pipeline failing at SonarQube stage — blocking all deployments.

Steps taken:

- Checked SonarQube dashboard — quality gate failing due to new critical security vulnerability.
- Identified the vulnerable dependency in pom.xml.
- Coordinated with dev team to update dependency version to patched release.
- Re-ran pipeline — quality gate passed.
- Added SonarQube dependency check to development process to catch earlier.

Challenge 4: ArgoCD OutOfSync — Manual Change in Production

Problem: ArgoCD showing Production app as OutOfSync after an emergency manual change.

Steps taken:

- ran: argocd app diff prod-app — showed manual scaling change in Deployment.
- Confirmed the manual change was intentional (emergency scaling during peak load).
- Updated the manifest in Git to match the emergency change.
- ArgoCD synced — cluster and Git back in sync.
- Raised incident review — created runbook for emergency scaling that routes through Git to prevent future drift.

Challenge 5: Terraform State Lock Stuck

Problem: Team member's terraform apply crashed mid-run — DynamoDB lock not released. No one else could run Terraform.

Steps taken:

- Checked DynamoDB table — found stale lock entry with crashed session ID.
- Verified the previous apply was truly dead (checked with the team member).
- Ran: terraform force-unlock — released the stale lock.
- Ran terraform plan to verify state was consistent before proceeding.
- Added documentation to runbook for handling stale locks.

SECTION 3: AWS

Beginner

What is a VPC and why do we need it?

Answer:

VPC (Virtual Private Cloud) is your own isolated private network inside AWS. Without a VPC, all your resources would be on a shared public network — any other AWS customer could potentially reach them.

VPC lets you define your own IP range, subnets, routing rules, and security controls.

In our project: we created a VPC with CIDR 10.0.0.0/16, split into public subnets (for ALB and NAT Gateway) and private subnets (for EKS nodes and RDS).

Tip:

Think of VPC as your own private data center inside AWS.

Beginner

What is the difference between Security Groups and NACLs?

Answer:

Security Groups: instance-level firewall. Stateful — if you allow inbound port 443, the response traffic is automatically allowed out. Applied to EC2, RDS, EKS nodes.

NACLs (Network Access Control Lists): subnet-level firewall. Stateless — you must explicitly allow both inbound AND outbound for each rule.

In our project: ALB Security Group allowed inbound 443 from internet. EKS node Security Group allowed only from ALB Security Group. RDS Security Group allowed only from EKS Security Group. NACLs added an additional subnet-level defence layer.

Watch out:

NACL is stateless — forgetting to add outbound rules is a common mistake that breaks connectivity.

Intermediate

Explain your multi-AZ VPC architecture.

Answer:

We designed VPC with 2 Availability Zones, each having a public and private subnet — 4 subnets total.

Public subnets (AZ-1: 10.0.1.0/24, AZ-2: 10.0.2.0/24): ALB and NAT Gateways. Route table: 0.0.0.0/0 → Internet Gateway.

Private subnets (AZ-1: 10.0.3.0/24, AZ-2: 10.0.4.0/24): EKS worker nodes and RDS. Route table: 0.0.0.0/0 → NAT Gateway.

Multi-AZ benefit: if AZ-1 goes down, ALB and EKS continue serving traffic from AZ-2. RDS Multi-AZ automatically fails over to standby in AZ-2.

NAT Gateway per AZ: if AZ-1 NAT fails, private resources in AZ-2 still have outbound connectivity.

Intermediate

Why ALB over NLB? How does path-based routing work?

Answer:

ALB operates at Layer 7 (HTTP/HTTPS). NLB operates at Layer 4 (TCP). We chose ALB because our microservices needed path-based routing.

Path-based routing: ALB listener rule — if path is /api/orders → forward to orders-service target group. If /api/users → users-service target group. NLB cannot do this.

ALB also integrates with ACM (AWS Certificate Manager) for SSL — SSL terminates at ALB, no manual cert installation on application servers.

ALB health checks every 30s — if pod fails, ALB stops sending traffic automatically.

In our project: ALB in public subnets, target groups pointing to EKS NodePort services.

Intermediate

What is IRSA and why is it better than IAM access keys?

Answer:

IRSA = IAM Roles for Service Accounts. Allows Kubernetes pods on EKS to assume an AWS IAM role and get temporary credentials automatically.

How it works: Create IAM role with required permissions → annotate K8s ServiceAccount with role ARN → EKS OIDC provider issues temporary credentials to the pod at startup.

Why better than access keys: access keys are long-lived — if leaked in code or Docker image, they give permanent AWS access. IRSA credentials are temporary (expire every hour) and automatically rotated.

In our project: ECR push role assigned via IRSA to CI pods. S3 read role assigned to application pods needing artifact access. Each pod only gets the minimum permissions it needs.

Tip:

IRSA is the AWS-recommended approach for EKS. Access keys in pods/Secrets is an anti-pattern.

Advanced

How did you handle RDS Multi-AZ? What happens during failover?

Answer:

RDS Multi-AZ runs a synchronous standby replica in a different AZ. AWS handles replication automatically.

During failover (AZ failure, hardware issue, or manual reboot): AWS updates the RDS DNS endpoint to point to the standby. Failover takes 60–120 seconds. Application reconnects automatically because it uses the same DNS endpoint.

We used MySQL Multi-AZ for BT Group's migration project. Connection string in K8s Secret uses the RDS endpoint DNS — not a hardcoded IP — so failover is transparent to the application.

Standby is NOT used for reads. For read scaling, you would use Read Replicas (separate feature). Multi-AZ is purely for HA failover.

Watch out:

Multi-AZ ≠ Read Replicas. Very common mistake. Multi-AZ = HA. Read Replicas = read scaling.

Advanced

How did you achieve ~15% AWS cost reduction?

Answer:

Three areas: EC2 right-sizing, Reserved Instances, and S3 lifecycle policies.

Right-sizing: Analysed CloudWatch metrics over 4 weeks. Found instances running at 10–15% CPU utilisation. Downsized over-provisioned instances to smaller types.

Reserved Instances: Jenkins, SonarQube, and Nexus run 24/7 predictably. Switched from On-Demand to 1-year Reserved Instances — ~40% cheaper for same instance type.

S3 lifecycle policies: Build artifacts transitioned to S3 Glacier after 30 days, deleted after 90 days. Old artifacts do not need expensive Standard storage.

Scenario

A pod cannot reach the RDS database. How do you troubleshoot?

Answer:

Step 1: `kubectl exec` into the pod, try: `telnet 3306` — check basic connectivity.

Step 2: If connection refused — check RDS Security Group. Does it allow port 3306 from the EKS node Security Group?

Step 3: Check NACLs — are both inbound and outbound rules allowing the traffic?

Step 4: Check the K8s Secret holding DB connection string — is the RDS endpoint correct?

Step 5: Check RDS status in AWS console — is it available? If Multi-AZ failover happened, endpoint should still resolve but verify.

Step 6: Check CloudWatch RDS metrics — DatabaseConnections count, any errors.

Scenario

How does traffic reach a pod in your AWS setup end-to-end?

Answer:

User → Route 53 DNS resolves domain to ALB DNS name.

→ Internet Gateway (IGW) — entry point into VPC public subnet.

→ ALB (public subnet) — SSL termination via ACM cert, path-based routing to target group.

→ EKS Worker Node (private subnet) — ALB forwards to NodePort of K8s Service.

→ Kubernetes Service (ClusterIP) — selects healthy pods via label selectors.

→ Pod — application handles request, queries RDS via JDBC.

Outbound from pod (for ECR image pull): Pod → NAT Gateway → IGW → ECR endpoint.

SECTION 4: Terraform

Beginner

What is Terraform and what problem does it solve?

Answer:

Terraform is an Infrastructure as Code (IaC) tool — you define infrastructure in code (.tf files) and Terraform creates/manages it. Problem it solves: manual infrastructure creation is error-prone, not repeatable, and hard to track. With Terraform, the same code creates identical infrastructure every time. Changes are tracked in version control. Team members can review infrastructure changes like code reviews.

In our project: we used Terraform to provision entire AWS environment — VPC, EKS, RDS, ALB, ECR, IAM — consistently across Dev, Staging, and Production.

Beginner

What is the difference between terraform plan and terraform apply?

Answer:

terraform plan: reads current state and desired state (your .tf files), calculates what changes need to be made, shows you the diff — without making any actual changes. Like a dry run.

terraform apply: executes the changes shown in plan. Creates, modifies, or destroys resources.

Best practice: always run plan first, review carefully, then apply. In our project: plan output was reviewed by senior before any apply in staging or production environments.

Intermediate

How does Terraform remote state work with S3 and DynamoDB?

Answer:

Local state (default): state file stored on your laptop — problem if working in a team.

Remote state: state file stored in S3 — all team members read/write the same state. S3 versioning enabled for recovery.

DynamoDB state locking: when anyone runs plan/apply, Terraform writes a lock to DynamoDB. If someone else tries to apply simultaneously, they get a lock error — preventing state corruption.

Our backend config: `backend 's3' { bucket='btgroup-tf-state', key='env/dev/terraform.tfstate', region='ap-south-1', dynamodb_table='btgroup-tf-locks' }`

Separate state file per environment — dev, staging, prod — so dev changes never touch prod state.

Watch out:

If DynamoDB lock gets stuck (crashed apply): run terraform force-unlock after confirming the previous apply is truly dead.

Intermediate

How did you structure reusable Terraform modules?

Answer:

Each module is a folder with: main.tf (resources), variables.tf (inputs), outputs.tf (outputs for other modules to consume).

We created modules for: VPC, EKS, RDS, ALB. Each module accepted environment-specific variables.

Root module called child modules with different .tfvars files per environment: `terraform apply -var-file=dev.tfvars`

Benefit: fix a bug in VPC module once — it fixes all environments. New environment = new .tfvars file, no new code.

Example: VPC module accepted: `environment_name`, `vpc_cidr`, `azs`, `private_subnet_cidrs`, `public_subnet_cidrs` as variables.

Advanced

What is Terraform state? What happens if state gets corrupted?

Answer:

State file (.tfstate) maps Terraform resource definitions to real-world resource IDs. Without state, Terraform doesn't know which AWS resource corresponds to which code block.

If state gets corrupted or deleted: Terraform loses track of existing resources. Running apply would try to create duplicate resources.

Recovery: S3 versioning lets you restore previous state version. Use `terraform state pull` to download current state, fix manually, `terraform state push` to upload.

`terraform state list`: shows all resources in state. `terraform state rm`: removes resource from state tracking without deleting the real resource. `terraform import`: imports existing resource into state.

Scenario

Terraform apply fails halfway through. What do you do?

Answer:

First: check error message — what resource failed and why (permissions, quota, naming conflict).

Fix the underlying issue (e.g., increase EC2 limit, fix IAM permission, rename conflicting resource).

Run terraform plan again — Terraform shows only remaining changes (successfully created resources are in state, failed ones are not).

Run terraform apply — only the failed resources are retried. Already-created resources are not touched (idempotency).

If partially created resources are in bad state: may need to terraform destroy specific resource and recreate.

Scenario

How do you manage secrets in Terraform without hardcoding?

Answer:

Never hardcode passwords in .tf files or .tfvars — they end up in state file and Git history.

Options we used: AWS Secrets Manager or SSM Parameter Store for sensitive values. Reference in Terraform using data source: data 'aws_secretsmanager_secret_version'.

For RDS password: generated using random_password resource, stored in Secrets Manager, referenced by application via IRSA — never in plain text.

Terraform state encryption: S3 bucket with SSE (server-side encryption) for state file at rest.

SECTION 5: Kubernetes

Beginner

What is a Pod? How is it different from a container?

Answer:

A container is a running Docker image — an isolated process.

A Pod is the smallest deployable unit in Kubernetes. A pod wraps one or more containers that share the same network namespace (same IP, same localhost) and storage volumes.

Most pods have one container. Multi-container pods are used for sidecar patterns (e.g., log shipper alongside the main app).

Pods are ephemeral — they can be killed and rescheduled on any node. That is why you need Services (stable IP) and PersistentVolumes (stable storage).

Beginner

What is the difference between Deployment, Service, and Ingress?

Answer:

Deployment: manages the desired number of pod replicas. Handles rolling updates and rollbacks. 'Run 3 replicas of this pod, always.'

Service: provides a stable IP and DNS name for a set of pods. Pods die and get new IPs — Service IP stays constant. Types: ClusterIP (internal), NodePort (exposes on node IP), LoadBalancer (creates cloud LB).

Ingress: HTTP/HTTPS routing rules at Layer 7. Routes traffic to different Services based on URL path or hostname. Requires an Ingress Controller (like NGINX or AWS Load Balancer Controller) to be installed.

Intermediate

Walk me through how you upgrade a Kubernetes node without downtime.

Answer:

Step 1: `kubectl cordon` — marks node unschedulable. No new pods placed here.

Step 2: `kubectl drain --ignore-daemonsets --delete-emptydir-data` — evicts all pods gracefully. K8s reschedules them on other nodes.

Step 3: Verify pods moved: `kubectl get pods -o wide` — no application pods on drained node.

Step 4: Perform OS patching or K8s version upgrade on the node.

Step 5: `kubectl uncordon` — node rejoins cluster as schedulable.

Step 6: Verify: `kubectl get nodes` — node shows correct version and Ready status.

This process ensures zero downtime because pods are gracefully moved before any changes happen.

Watch out:

Always cordon BEFORE drain. Wrong order can cause downtime. Never skip uncordon after patching.

Intermediate

How does Kubernetes certificate lifecycle work in kubeadm?

Answer:

kubeadm generates a PKI (Certificate Authority) for the cluster. All control plane components (API server, etcd, controller-manager, scheduler) communicate via mTLS using certs signed by this CA.

Cert expiry: by default, kubeadm certs expire every 1 year.

Check expiry: `kubeadm certs check-expiration` — shows all certs and expiry dates.

Renew: `kubeadm certs renew all` — renews all control plane certificates.

After renewal: restart static pods — delete pod manifests under `/etc/kubernetes/manifests` and kubelet recreates them, OR restart kubelet.

Update kubeconfig: copy new `/etc/kubernetes/admin.conf` for `kubectl` access.

We tracked expiry 30 days in advance and renewed proactively — zero unplanned expiry incidents.

Intermediate

What is RBAC in Kubernetes? How did you implement it?

Answer:

RBAC = Role-Based Access Control. Controls who can do what in the Kubernetes API.

Four objects: Role (permissions within a namespace), ClusterRole (permissions cluster-wide), RoleBinding (assigns Role to user/ServiceAccount in a namespace), ClusterRoleBinding (assigns ClusterRole cluster-wide).

In our project: Dev team had Role with `get/list/watch` pods/logs in their namespace — no exec, no secrets access. Ops team had ClusterRole with broader permissions. Production namespace: read-only for most, write-only for the CI service account used by

ArgoCD.

ServiceAccount RBAC: ArgoCD ServiceAccount had ClusterRole to apply manifests. Jenkins ServiceAccount had permission to update image tags in manifests repo.

Advanced

How does a pod crash get detected and what is the self-healing process?

Answer:

Kubelet on each node monitors the containers it runs. If a container exits with non-zero code, kubelet restarts it (per restartPolicy). CrashLoopBackOff: container keeps crashing. Kubelet uses exponential backoff before retrying — 10s, 20s, 40s... up to 5 minutes between restarts.

Liveness Probe: kubelet sends HTTP/exec probe to container. If probe fails N consecutive times — kubelet kills and restarts the container.

Readiness Probe: if readiness fails — pod is removed from Service endpoints. Traffic stops going to it. Pod stays running — not killed.

Node failure: if node stops sending heartbeats to API server for 5 minutes (default), node is marked NotReady. Controller Manager evicts pods and reschedules on healthy nodes.

Scenario

How do you debug a pod stuck in Pending state?

Answer:

kubectl describe pod — check Events section.

Common causes: Insufficient resources (CPU/memory) on all nodes — scheduler cannot place pod. Fix: scale up nodes or reduce resource requests.

Node selector / affinity not matching any node — check nodeSelector or affinity rules in pod spec.

PVC not bound — if pod needs PersistentVolumeClaim that isn't bound, pod stays Pending. Check: kubectl get pvc -n .

Image pull failure — if pod needs image and imagePullPolicy is Always but registry unreachable — may show as Pending briefly before showing ErrImagePull.

Taints and tolerations — node has a taint, pod doesn't have matching toleration.

Scenario

ArgoCD shows app as OutOfSync. How do you handle it?

Answer:

Step 1: argocd app diff — shows exactly what is different between Git and live cluster.

Step 2: Identify cause — manual kubectl apply (most common), horizontal pod autoscaler changed replica count, manual config edit.

Step 3: Determine correct state — if Git is correct: argocd app sync — cluster updates to match Git.

If manual change was intentional (e.g., emergency scaling): update Git manifest first, then sync.

Step 4: Prevention — enforce policy that no manual kubectl apply in production. All changes through Git. Configure ArgoCD self-heal to auto-sync on drift.

Architecture

kubeadm vs EKS — when would you use each?

Answer:

kubeadm (self-managed): you manage everything — control plane VMs, etcd, cert rotation, master upgrades, HA setup. More control, more operational burden. Used when you cannot use cloud managed services (on-prem, air-gapped, specific compliance).

EKS (managed): AWS manages control plane — no cert rotation for masters, no etcd backup, no master node patching. You manage worker nodes and workloads only. Reduces operational overhead significantly.

In our project: production was kubeadm (existing on-prem infrastructure). AWS migration used EKS — aligned with cloud-native approach and reduced ops burden.

Cost: EKS control plane costs \$0.10/hour (~\$73/month). For on-prem, you pay for the master VMs but no per-cluster fee.

SECTION 6: Docker

Beginner

What is the difference between a Docker image and a container?

Answer:

Image: a read-only template — like a blueprint. Contains application code, runtime, dependencies, config. Built from a Dockerfile. Stored in a registry (Harbor/ECR).

Container: a running instance of an image. Writable layer added on top of read-only image. Multiple containers can run from the same image.

Analogy: image = class definition. Container = object (instance) of that class.

Beginner

What is a Dockerfile? Walk me through a basic one.

Answer:

Dockerfile is a text file with instructions to build a Docker image.

FROM openjdk:17-jre-slim — base image (Java runtime).

WORKDIR /app — sets working directory inside container.

COPY target/app.jar /app/app.jar — copies JAR from build into image.

EXPOSE 8080 — documents the port (does not publish it — that happens at docker run).

CMD ['java', '-jar', 'app.jar'] — command to run when container starts.

In our project: Dockerfile in the repo. Jenkins runs `docker build -t harbor.btgroup.com/app:${VERSION}` . then `docker push`.

Intermediate

What is a multi-stage Dockerfile and why use it?

Answer:

Multi-stage build: multiple FROM statements in one Dockerfile. Each stage is a separate image.

Stage 1 (build): FROM maven:3.9 — runs mvn package to build the JAR. This stage has all build tools.

Stage 2 (runtime): FROM openjdk:17-jre-slim — copies only the JAR from stage 1. No Maven, no source code, no build tools in final image.

Result: final image is much smaller (only runtime needed, not build tools). Smaller image = faster pull, smaller attack surface, less storage in registry.

In our project: we used multi-stage for Spring Boot apps. Final image ~200MB instead of ~600MB with full Maven.

Intermediate

How do Docker networking modes work?

Answer:

bridge (default): containers get their own IP on a virtual bridge network. Containers communicate by container name. Host machine can reach containers via port mapping (`-p 8080:8080`).

host: container shares the host's network namespace. Container uses host IP directly. No port mapping needed. Less isolation.

none: no networking. Container completely isolated.

overlay: for Docker Swarm — cross-host networking. Not used in K8s (K8s has its own CNI networking).

In Kubernetes: Docker networking is replaced by the CNI plugin (Calico/Flannel). Each pod gets its own IP. Container-to-container in same pod: via localhost.

Scenario

Docker image build fails. How do you troubleshoot?

Answer:

Read the error — which line in Dockerfile failed.

RUN command failed: check if package exists, if base image has required tools, if network is accessible for package download.

COPY failed: file doesn't exist at the path specified — check `.dockerignore` isn't excluding it, check relative path.

Permission denied: file permissions issue. Add `RUN chmod +x /app/script.sh` before the COPY or after.

`docker build --no-cache`: if caching is serving stale layers, rebuild from scratch.

`docker run -it /bin/bash`: run base image interactively, manually run the failing command to see detailed error.

Advanced

How do you reduce Docker image size?

Answer:

Use minimal base image: openjdk:17-jre-slim instead of openjdk:17. Alpine-based images even smaller.

Multi-stage builds: only copy final artifact to runtime image.

Combine RUN commands: each RUN creates a layer. RUN apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/* — one layer, cleanup in same layer.

Use .dockerignore: exclude node_modules, .git, test files from build context.

Don't install unnecessary packages. Remove package manager cache in same RUN layer.

SECTION 7: Jenkins

Beginner

What is Jenkins and what is a Jenkinsfile?

Answer:

Jenkins is an open-source CI automation server. It listens for code changes (via webhooks or polling) and triggers build pipelines automatically.

Jenkinsfile: a text file (Groovy DSL) in the root of the repo that defines the pipeline stages. Pipeline as code — version controlled, reviewable.

In our project: developer merges to main → GitLab webhook hits Jenkins → Jenkins reads Jenkinsfile → runs stages: Checkout → Maven Build → SonarQube → Nexus → Docker → ArgoCD trigger.

Intermediate

Explain a declarative Jenkins pipeline with stages.

Answer:

pipeline { agent any — run on any available agent.

stages { stage('Build') { steps { sh 'mvn clean package' } } }

stage('SonarQube') { steps { withSonarQubeEnv('SonarQube') { sh 'mvn sonar:sonar' } } }

stage('Quality Gate') { steps { waitForQualityGate abortPipeline: true } } }

stage('Docker Build') { steps { sh 'docker build -t harbor.btgroup.com/app:\${BUILD_NUMBER}' } }

stage('Push') { steps { sh 'docker push harbor.btgroup.com/app:\${BUILD_NUMBER}' } }

post { always { cleanWs() } failure { mail to: 'team@btgroup.com', subject: 'Build Failed' } } }

waitForQualityGate: Jenkins waits for SonarQube to process results and aborts if gate fails.

Intermediate

How do you handle credentials in Jenkins?

Answer:

Jenkins Credentials Store: store secrets (passwords, API keys, SSH keys) encrypted in Jenkins. Reference in pipeline using credentials() binding — never hardcoded in Jenkinsfile.

Example: withCredentials([usernamePassword(credentialsId: 'nexus-creds', usernameVariable: 'USER', passwordVariable: 'PASS')]) { sh 'docker login -u \$USER -p \$PASS harbor.btgroup.com' }

In our project: Harbor credentials, SonarQube token, GitLab token all stored in Jenkins Credentials. CyberArk integration for server SSH credentials via Ansible AWX.

Watch out:

Never echo credentials in shell steps — they will appear in build logs.

Advanced

What is a shared library in Jenkins?

Answer:

Shared library: reusable Groovy code stored in a separate Git repo, loaded into Jenkins pipelines.

If 20 pipelines all do the same Docker build steps — instead of copying code 20 times, extract into shared library. One change fixes all pipelines.

In vars/ folder: define global functions (e.g., buildAndPush.groovy). Call in Jenkinsfile: @Library('my-shared-lib') _ → buildAndPush('harbor.btgroup.com/app').

In our project: we used shared library for common stages — Docker build, push to Harbor, update ArgoCD manifest — standardized across all application pipelines.

Scenario

Jenkins pipeline is slow. How do you optimize it?

Answer:

Parallel stages: run SonarQube scan and unit tests in parallel instead of sequential.

Docker layer caching: use --cache-from in docker build to reuse unchanged layers.

Nexus caching: configure Maven settings.xml to use Nexus as mirror — downloads dependencies from local Nexus instead of Maven Central.

Agent labels: use specific agents with pre-installed tools instead of installing tools on every run.

Lightweight checkout: use sparse checkout if repo is large.

Clean workspace only when needed — not every build.

Scenario

A Jenkins build fails at the Docker push stage. How do you debug?

Answer:

Check build log for exact error message.

Authentication failure: Harbor/ECR credentials expired or wrong. Re-check Jenkins credentials store. For ECR: aws ecr get-login-password and check IAM permissions.

Network issue: Jenkins agent cannot reach Harbor/ECR endpoint. Check firewall rules, proxy settings.

Image tag conflict: if using same tag and registry rejects overwrite — use unique tags (BUILD_NUMBER or git commit hash).

Disk space: Jenkins agent disk full. Clean up old Docker images: docker image prune -a.

SECTION 8: ArgoCD & GitOps

Beginner

What is GitOps? Why is it better than traditional CD?

Answer:

GitOps: Git is the single source of truth for both application code AND cluster state. Every change to infrastructure/deployment must come through a Git commit. No manual kubectl apply in production.

ArgoCD watches the Git manifest repo. When it detects a change, it reconciles the live cluster to match Git.

Why better: every deployment is auditable (Git history). Rollback = revert a commit. Drift detection — any manual change is flagged. Team review via pull requests before anything hits cluster.

Watch out:

Traditional CD: pipeline pushes to cluster directly. No audit trail. No drift detection. Rollback requires re-running old pipeline.

Intermediate

What is OutOfSync in ArgoCD? How do you fix it?

Answer:

OutOfSync: live cluster state differs from what Git manifest says it should be.

Common causes: manual kubectl apply in cluster, HPA changed replica count, ConfigMap edited manually.

Debug: argocd app diff — shows exactly what is different.

Fix option 1: if Git is correct — argocd app sync — cluster updated to match Git.

Fix option 2: if manual change was intentional — update Git first to reflect the change, then sync.

Prevention: enforce no direct kubectl apply in prod. Use ArgoCD self-heal: auto-sync enabled, prune=true, selfHeal=true in Application spec.

Intermediate

How do you rollback a deployment using ArgoCD?

Answer:

Method 1 (Git revert): Revert the commit that changed the image tag in manifest repo. ArgoCD detects revert and syncs back to previous image. Full audit trail maintained.

Method 2 (ArgoCD UI/CLI): argocd app rollback — rolls back to a previous sync revision. Does NOT update Git — cluster will show as OutOfSync after rollback.

Best practice: Method 1 (Git revert) is the GitOps-correct approach. Everything tracked in Git.

In our project: we used Git revert for rollbacks — ArgoCD detected and synced automatically within 3 minutes.

Advanced

What is the difference between ArgoCD sync and self-heal?

Answer:

Sync: manually triggered (or automated) reconciliation of cluster to match current Git state.

Self-heal: when enabled, ArgoCD continuously watches the live cluster. If ANY manual change is made to a managed resource, ArgoCD AUTOMATICALLY reverts it back to Git state.

Without self-heal: manual changes stick until next sync. With self-heal: manual changes are reverted within seconds.

In our project: self-heal enabled in Production — enforces that ALL changes go through Git. Non-production had self-heal disabled — allows ops team to make quick manual changes for testing.

Scenario

How does ArgoCD handle multiple environments (Dev/Staging/Prod)?

Answer:

Separate ArgoCD Application resources per environment. Each Application points to: same manifest repo but different directory or branch, and different target cluster/namespace.

Example: App 'dev-app' → manifest repo /environments/dev → dev cluster. App 'prod-app' → manifest repo /environments/prod → prod cluster.

Promotion: update image tag in /environments/staging → ArgoCD syncs staging. After testing, update /environments/production → ArgoCD syncs prod. Manual or automated promotion.

In our project: 4 ArgoCD Applications for 4 environments. Promotion was manual — ops team updated manifest after staging sign-off.

SECTION 9: Helm

Beginner

What is Helm and what problem does it solve?

Answer:

Helm is a package manager for Kubernetes. A Helm chart is a collection of Kubernetes manifest templates packaged together.

Problem: deploying a complex app to K8s requires multiple YAML files (Deployment, Service, ConfigMap, Secret, Ingress).

Managing these separately across environments is tedious.

Helm packages them into one chart. Values (image tag, replica count, resource limits) are variables in a values.yaml file — different values per environment, same chart.

In our project: Helm is in our tech stack. Our primary deployment method is GitOps via ArgoCD with raw manifests. Helm knowledge is important for community charts we use (e.g., prometheus-community Helm chart for Prometheus installation).

Intermediate

What is the structure of a Helm chart?

Answer:

Chart.yaml: chart metadata — name, version, description.

values.yaml: default values for template variables.

templates/: folder with Kubernetes manifest templates using Go templating. Example: image: {{ .Values.image.repository }}:{{ .Values.image.tag }}

templates/NOTES.txt: printed to user after install.

_helpers.tpl: reusable template definitions.

Deploy: helm install my-release ./my-chart -f prod-values.yaml. Upgrade: helm upgrade my-release ./my-chart -f prod-values.yaml. Rollback: helm rollback my-release 1.

Intermediate

What is a Helm release and how do you manage it?

Answer:

A release is a deployed instance of a chart in a cluster. Same chart can be installed multiple times with different release names.

helm list: shows all releases. helm history : shows upgrade history. helm status : current state.

helm upgrade --install: installs if not exists, upgrades if exists — idempotent, good for CI/CD.

helm rollback : rolls back to previous chart version. Revision numbers shown in helm history.

Scenario

Helm upgrade fails. How do you troubleshoot?

Answer:

helm status — shows current state (failed, deployed, pending).

helm history — shows all revisions and their status.

If upgrade failed and release is in failed state: helm rollback .

Check pod events: kubectl describe pod — see if new pods are failing readiness/liveness.

Dry run: helm upgrade --dry-run --debug — shows rendered manifests and any template errors without applying.

helm get manifest : shows what Helm actually deployed to cluster.

SECTION 10: Ansible AWX

Beginner

What is Ansible and what is AWX?

Answer:

Ansible is an agentless automation tool — uses SSH to connect to servers and execute tasks defined in YAML playbooks. No agent installed on target servers.

AWX is the open-source web UI and API for Ansible. It provides: job scheduling, RBAC for who can run which playbooks, credential management, job templates, and audit logs.

In our project: AWX used for Apache Tomcat deployment automation and OS patch management across 110+ Linux servers.

Intermediate

Explain your Ansible AWX + CyberArk integration.

Answer:

Challenge: AWX needs SSH credentials to connect to 110+ servers. Hardcoding passwords is a PCI DSS violation.

Solution: CyberArk integration. AWX Credential Type configured to fetch SSH credentials from CyberArk vault at job runtime — password never stored in AWX.

Flow: AWX job starts → calls CyberArk API to fetch SSH password for target server → uses password for SSH connection → CyberArk logs the access (audit trail for PCI DSS).

Result: zero hardcoded credentials anywhere. Every privileged access auditable in CyberArk. PCI DSS compliant.

Intermediate

What is an Ansible playbook? Walk me through your Tomcat deployment playbook.

Answer:

Playbook: YAML file defining tasks to execute on target hosts.

Our Tomcat deployment playbook: hosts: tomcat-servers — inventory group of target servers.

Task 1: Stop Tomcat service — systemd module: service: name=tomcat state=stopped.

Task 2: Backup current WAR — copy current webapp to backup directory.

Task 3: Deploy new WAR — fetch artifact from Nexus, copy to webapps/ directory.

Task 4: Start Tomcat — service: name=tomcat state=started.

Task 5: Health check — uri module: wait_for URL to return HTTP 200.

Rollback: if health check fails, task triggers rollback — restores backup WAR and restarts Tomcat.

Result: ~60% reduction in deployment effort vs manual SSH-based deployment.

Advanced

What is idempotency in Ansible? Why does it matter?

Answer:

Idempotency: running the same playbook multiple times produces the same result. No side effects from re-runs.

Ansible modules are designed to be idempotent. Example: file module with state=present creates the file if not present — does nothing if already present. Does not create duplicate.

Matters because: CI/CD runs playbooks repeatedly. If not idempotent — running twice could break things (e.g., create duplicate users, duplicate config entries).

Non-idempotent example (bad): shell: echo 'config=value' >> /etc/app.conf — appends every time. Fix: use lineinfile module which checks before adding.

Scenario

Ansible playbook fails on 3 out of 20 servers. What do you do?

Answer:

Ansible continues by default to remaining hosts after a failure (unless serial or any_errors_fatal is set).

After run: check output — Ansible shows which hosts failed with error message.

SSH issues: check if AWX can reach those 3 servers — network/firewall change?

Disk space: deployment task failed because server disk is full.

Service state: Tomcat was already in wrong state — service not stopping cleanly.

Re-run on failed hosts only: ansible-playbook -i inventory playbook.yml --limit failed-server1,failed-server2

Fix underlying issue first, then re-run.

SECTION 11: Linux

Beginner

What are the most common Linux commands you use daily?

Answer:

File operations: ls -la, cd, cp, mv, rm, mkdir, find /path -name '*.log' -mtime +7

File content: cat, tail -f /var/log/app.log, grep 'ERROR' app.log, less, head

Process management: ps aux | grep java, top, htop, kill -9, systemctl status tomcat

Disk/Memory: df -h (disk usage), free -m (memory), du -sh /var/log (folder size)

Network: netstat -tulpn (listening ports), ss -tulpn, curl -v http://localhost:8080, telnet

Permissions: chmod 755, chown user:group, ls -la

Intermediate

A Linux server is running slow. How do you investigate?

Answer:

Step 1: top or htop — which process is consuming most CPU?

Step 2: free -m — is memory exhausted? Is there heavy swap usage?

Step 3: df -h — is disk full? Full disk causes many applications to hang.

Step 4: iostat -x 1 — is disk I/O at 100%? Which disk?

Step 5: netstat -tulpn — too many connections? Connection in CLOSE_WAIT (connection leak)?

Step 6: tail -f /var/log/syslog or journalctl -f — any kernel errors, OOM killer events?

Step 7: uptime — load average. If load > number of CPUs, system is overloaded.

Intermediate

What is a systemd service? How do you manage one?

Answer:

systemd is the init system on modern Linux (RHEL 7+, Ubuntu 16+). Manages services, daemons, logging.

systemctl status tomcat — current state of service.

systemctl start/stop/restart tomcat — control service.

systemctl enable tomcat — start on boot.

systemctl disable tomcat — don't start on boot.

journalctl -u tomcat -f — live logs for a specific service.

In our project: Tomcat managed via systemd. Ansible playbooks use systemd module for start/stop.

Advanced

How do you diagnose a network connectivity issue between two servers?

Answer:

ping : basic ICMP reachability.

telnet : can we reach specific port? Confirms TCP connectivity.

traceroute : shows network hops — where does it drop?

netstat -tulpn on target: is the service actually listening on the port?

iptables -L on both servers: any firewall rule blocking traffic?

tcpdump -i eth0 port 3306: capture packets to see if traffic is reaching the server.

Check security groups (AWS) or firewall rules on VMs.

Scenario

Disk space is 100% on a server. What do you do?

Answer:

df -h: confirm which filesystem is full.

du -sh /var/log/* | sort -rh | head: find largest directories.

find /var/log -name '*.log' -mtime +30 -delete: delete old log files.

journalctl --vacuum-size=500M: truncate systemd journal.

Docker: docker system prune -a: remove unused images, stopped containers.

Kubernetes: old pod log files in /var/log/pods/.

Raise long-term fix: add log rotation (logrotate), increase volume size, add monitoring alert for >80% disk usage.

SECTION 12: Git (GitHub / GitLab)

Beginner

What is the difference between git pull and git fetch?

Answer:

git fetch: downloads changes from remote but does NOT merge them into your local branch. Safe — doesn't change your working files.

git pull: git fetch + git merge in one step. Downloads and merges remote changes into current branch.

When to use fetch: when you want to see what changed on remote before merging. git fetch origin → git diff main origin/main.

When to use pull: when you want to update your local branch with remote changes.

Intermediate

What is the difference between git merge and git rebase?

Answer:

git merge: combines two branches by creating a merge commit. Preserves full history — you can see when branches diverged and joined.

git rebase: moves your branch commits on top of the target branch. Linear history — no merge commits. Cleaner log but rewrites commit history.

In our project: feature branches merged via Merge Requests in GitLab (git merge with --no-ff). Rebase used locally to clean up commits before raising MR.

Rule: never rebase shared branches (main/develop). Only rebase your own feature branches before merging.

Tip:

In interviews: mention you used merge for team collaboration (MRs) and rebase for local cleanup.

Watch out:

git rebase on a shared branch that others are using = disaster (rewrites shared history).

Intermediate

How do webhooks work between GitLab and Jenkins?

Answer:

GitLab webhook: configured in GitLab project settings. When a push or merge event occurs, GitLab sends an HTTP POST to Jenkins URL.

Jenkins Generic Webhook Trigger or GitLab plugin: receives the POST, parses payload (branch name, commit hash), triggers the matching pipeline.

In our project: merge to main branch → GitLab sends webhook → Jenkins pipeline starts → checks out code → runs full pipeline.

Secret token: GitLab webhook configured with a secret token. Jenkins verifies the token — only GitLab can trigger builds (security).

Advanced

How do you handle a situation where you accidentally committed sensitive data to Git?

Answer:

Do NOT just delete the file in a new commit — the sensitive data is still in Git history.

If not pushed: git reset HEAD~1 — undo the commit. Remove the sensitive file. Re-commit without it.

If already pushed to shared repo: git filter-branch or BFG Repo Cleaner to rewrite history — removes file from all commits. Force push.

CRITICAL: Immediately rotate/revoke the exposed credentials — assume they are compromised once pushed. Git history on shared repos may be cached by mirrors.

Prevention: git-secrets or pre-commit hooks to scan for secrets before commit. Never commit .env files or credentials.

Scenario

What is a Git branching strategy? What did your team follow?

Answer:

Feature branch workflow: developers create feature branches from main. Raise MR when ready. After review + CI passes, merge to main.

In our project: main branch = production-ready code. Feature branches for each task/story. Hotfix branches for urgent production fixes.

GitLab CI ran on every MR — unit tests and SonarQube scan must pass before MR can be merged.

Protected branches: main and staging branches — only senior engineers / automated pipelines can push. Direct push by developers blocked.

SECTION 13: Maven

Beginner

What is Maven and what does it do?

Answer:

Maven is a build automation tool for Java projects. It handles: compiling source code, running unit tests, packaging into JAR/WAR, managing dependencies, and integrating with CI/CD.

pom.xml: Project Object Model — the configuration file. Defines: project dependencies (Spring Boot, JUnit etc.), build plugins (Surefire for tests, Dockerfile Maven plugin), and project metadata.

In our project: mvn clean package used in Jenkins pipeline. Downloads dependencies from Nexus (our internal mirror), compiles, tests, packages WAR/JAR.

Intermediate

What is the Maven build lifecycle?

Answer:

Key phases in order: validate → compile → test → package → verify → install → deploy.

validate: checks pom.xml is valid.

compile: compiles source code.

test: runs unit tests (Surefire plugin). Build fails if tests fail.

package: creates JAR or WAR based on in pom.xml.

install: installs artifact to local Maven repository (~/.m2).

deploy: uploads artifact to remote repository (Nexus in our case).

Running mvn package runs all phases up to and including package.

Intermediate

How did you configure Maven to use Nexus as a mirror?

Answer:

~/.m2/settings.xml (or Jenkins tool settings): configure section pointing to Nexus.

nexus*http://nexus.btgroup.com/repository/maven-public/

Why: Maven Central downloads are slow and unreliable in CI. Nexus acts as proxy — first time Maven downloads from Central and caches in Nexus. Subsequent builds pull from Nexus (fast, reliable, no external dependency).

Also: Nexus stores our internal artifacts (company JARs) — only accessible internally, never go to Maven Central.

Scenario

Maven build fails with 'dependency not found'. How do you fix it?

Answer:

Check pom.xml — is the dependency version correct? Check Maven Central for correct groupId/artifactId/version.

Check if Nexus is reachable — can Jenkins agent reach Nexus? Network/firewall issue?

Check Nexus repository — is the dependency in Nexus proxy cache? If Nexus hasn't cached it, it will try to download from Central.

mvn dependency:resolve -U: force Maven to re-download dependencies, bypassing local cache.

Clear local cache: rm -rf ~/.m2/repository/// — removes corrupted local cache.

Check pom.xml for snapshot vs release mismatch — SNAPSHOT versions must come from snapshot repository.

SECTION 14: Nexus Repository Manager

Beginner

What is Nexus and why is it used?

Answer:

Nexus is an artifact repository manager. It stores and manages build artifacts (JARs, WARs, Docker images, npm packages). Three repository types: Hosted (stores your internal artifacts), Proxy (caches external repositories like Maven Central), Group (combines hosted + proxy — single URL for Maven to use).

Why use it: Cache external dependencies locally — faster builds, no dependency on internet during CI. Store internal artifacts privately — not uploaded to public Maven Central. Version history — every build artifact stored and retrievable for rollback.

Intermediate

How does Jenkins push artifacts to Nexus?

Answer:

In pom.xml: section defines Nexus URL for releases and snapshots.

Jenkins runs mvn deploy: packages the artifact AND uploads to Nexus. Credentials configured in Maven settings.xml (referenced from Jenkins Credentials store — not hardcoded).

In our project: every successful build pushed to Nexus with version tag. Docker stage then fetched from Nexus to build image. Complete audit trail of which artifact version is in which Docker image.

Scenario

Nexus is down. What is the impact and how do you handle it?

Answer:

Impact: all Jenkins builds fail at dependency download stage. Cannot build any application.

Short-term: if local Maven cache (~/.m2) is warm on Jenkins agents, builds may still work for cached dependencies.

Recovery: restart Nexus service (systemctl restart nexus). Check disk space — Nexus fills disk if not managed. Check Nexus logs for startup errors.

Mitigation for future: Nexus HA setup or regular backups. Alert on Nexus service health via Prometheus.

SECTION 15: SonarQube

Beginner

What is SonarQube and what does it scan for?

Answer:

SonarQube is a static code analysis tool. It scans source code without executing it.

It detects: Bugs (logic errors), Vulnerabilities (security issues like SQL injection, hardcoded passwords), Code Smells (maintainability issues — too-long methods, duplicate code), Test Coverage (% of code covered by unit tests).

In our project: every Jenkins pipeline runs SonarQube scan after Maven build. Quality Gate must pass before Docker build proceeds.

Intermediate

What is a Quality Gate? What did yours look like?

Answer:

Quality Gate: a set of conditions. If ANY condition fails, the gate fails and pipeline stops.

Our Quality Gate conditions (example): Coverage $\geq$ 80% (unit test coverage), Duplicated Lines $<$ 3%, Critical Vulnerabilities = 0, Blocker Issues = 0.

Jenkins integration: withSonarQubeEnv() runs analysis. waitForQualityGate abortPipeline: true — Jenkins waits for SonarQube to process and aborts if gate fails.

Result: this enforced code quality standards across 20+ microservices — contributed to ~40% improvement in build consistency (fewer production bugs from quality issues).

Scenario

SonarQube quality gate fails. Developer says 'just skip it for now'. What do you do?

Answer:

Never skip — quality gate exists precisely to catch issues before production.

Work with developer to understand the failure: what specific rule triggered? Is it a false positive or a real issue?

If real issue: developer must fix it. No exceptions for critical vulnerabilities.

If false positive: SonarQube allows marking specific findings as 'Won't Fix' or 'False Positive' with a comment explaining why. This documents the decision.

If coverage drop: find which new code is not tested, add unit tests.

Process matters: bypassing quality gate today means production incidents tomorrow.

SECTION 16: Prometheus & Grafana

Beginner

What is Prometheus and how does it collect metrics?

Answer:

Prometheus is a time-series metrics database. It collects metrics by PULLING (scraping) from HTTP endpoints.

Applications expose metrics at /metrics endpoint. Prometheus scrapes this endpoint every N seconds (scrape_interval) and stores the data.

In Kubernetes: Prometheus scrapes: node-exporter (node CPU/memory/disk), kube-state-metrics (pod/deployment state), application /metrics endpoints.

Our setup: Prometheus installed via Helm chart (prometheus-community/kube-prometheus-stack). Scraped all K8s nodes and pods automatically via ServiceMonitor resources.

Intermediate

How did you configure alerting in Prometheus?

Answer:

Alerting rules defined in PrometheusRule resources (YAML). Example: alert if pod restart count > 5 in 10 minutes.

Alertmanager: receives fired alerts from Prometheus, routes to notification channels — email, Slack, PagerDuty.

In our project: alerts for pod CrashLoopBackOff, high CPU (>85%), low memory (<10% free), PVC nearly full, certificate expiry <30 days.

Alert → Alertmanager → Team email/Slack → engineer investigates → updates runbook with resolution.

This was key to achieving ~30% faster incident resolution — proactive alerts before users reported issues.

Intermediate

What is Grafana? How does it connect to Prometheus?

Answer:

Grafana is a visualization layer on top of data sources. It does NOT store data — it queries Prometheus (and other sources) and displays dashboards.

Connection: Grafana data source configured with Prometheus URL. Dashboards use PromQL (Prometheus Query Language) to query metrics.

Key dashboards we maintained: K8s cluster overview (node CPU/memory, pod count), Application dashboard (request rate, error rate, latency), CI/CD dashboard (build success rate, deployment frequency).

PromQL examples: rate(http_requests_total[5m]) — requests per second. container_memory_usage_bytes{pod=~'myapp.*'} — memory per pod.

Scenario

Alert fires: 'Pod memory usage > 90%'. How do you respond?

Answer:

Step 1: kubectl top pods -n — confirm which pod is at 90% memory.

Step 2: kubectl describe pod — check memory limit set in Deployment.

Step 3: kubectl logs — any memory leak signs? OutOfMemory errors?

Step 4: Check Grafana dashboard — is memory growing over time (leak) or constantly high (under-provisioned)?

If constantly high: increase memory limit in Deployment manifest (via Git → ArgoCD sync).

If growing over time: memory leak in application — alert dev team, schedule fix, consider pod restart as temporary mitigation.

Update runbook with steps taken.

SECTION 17: Dynatrace

Beginner

What is Dynatrace and how is it different from Prometheus?

Answer:

Dynatrace is an enterprise APM (Application Performance Monitoring) tool. It uses agents (OneAgent) installed on servers/nodes to automatically discover and monitor everything.

Dynatrace vs Prometheus: Prometheus scrapes metrics you define. Dynatrace auto-discovers services, traces requests end-to-end across microservices (distributed tracing), and uses AI (Davis AI) to detect anomalies and find root cause automatically.

In our project: Dynatrace OneAgent installed on all K8s nodes. It monitored: JVM performance, transaction traces across Spring Boot microservices, pod health, and triggered alerts for anomalies.

Intermediate

How did Dynatrace help with incident resolution?

Answer:

Dynatrace Davis AI automatically detects anomalies — no need to manually set thresholds for every metric.

Distributed tracing: when a request is slow, Dynatrace shows the full call chain — from user request → API gateway → microservice A → microservice B → database query. Shows exactly which service/query is slow.

Deployment events: Dynatrace detects deployments (image changes) and compares before/after metrics automatically — shows if a deployment caused performance regression.

In our project: integrated Dynatrace alerts with runbooks. Alert fires → engineer opens Dynatrace → Davis AI suggests root cause → engineer follows runbook → 30% faster resolution.

Scenario

Dynatrace shows a spike in response time after deployment. How do you investigate?

Answer:

Open Dynatrace → Services → find the affected service → Response time chart shows spike.

Click on a slow trace — Dynatrace shows full distributed trace: which service call took long.

Check if DB query time increased — slow query after deployment (code change affected query).

Check Deployment Events — compare response time before and after deployment.

If deployment is the cause: rollback via ArgoCD (revert Git commit). Alert dev team to investigate the slow query/code.

Dynatrace problem card: Davis AI creates a Problem record linking all related symptoms — use this for RCA documentation.

SECTION 18: SSL/TLS

Beginner

What is SSL/TLS and why is it important?

Answer:

SSL (Secure Sockets Layer) / TLS (Transport Layer Security) — cryptographic protocols that encrypt data in transit between client and server.

Why important: without TLS, data (passwords, payment info) travels in plaintext — anyone on the network can read it (man-in-the-middle attack).

Certificate: issued by a Certificate Authority (CA). Proves the server is who it claims to be. Contains: public key, domain name, expiry date, CA signature.

In our project: all external traffic HTTPS (port 443). VTM/ALB handles SSL termination. Java Keytool used for managing certificates on 110+ servers.

Intermediate

How did you manage SSL certificates using Java Keytool?

Answer:

Keystore: Java's format for storing private keys and certificates (.jks or .p12 file).

Truststore: stores trusted CA certificates — used for verifying incoming certificates in service-to-service communication.

Key Keytool commands:

keytool -list -v -keystore keystore.jks: view certificates and expiry dates.

keytool -importcert -file cert.pem -keystore truststore.jks: import CA cert.

keytool -genkeypair -alias myapp -keyalg RSA: generate new key pair.

In our project: I tracked expiry dates of 110+ server certificates. Raised Change Requests 30 days before expiry, renewed within CR window, verified services post-renewal.

Intermediate

What is the difference between SSL termination and SSL passthrough?

Answer:

SSL Termination at load balancer: LB decrypts HTTPS. Traffic from LB to backend is HTTP (plaintext internally). Simpler — backend doesn't need to handle SSL.

SSL Passthrough: LB forwards encrypted traffic without decrypting. Backend (pod/server) handles decryption. End-to-end encryption.

When to use passthrough: when compliance (PCI DSS) requires encryption all the way to the application — data must not be in plaintext even inside the data center.

In our project: most services used SSL termination at VTM. Specific PCI DSS-scoped services used SSL passthrough end-to-end.

Advanced

How did you handle the Kubernetes certificate expiry incident?

Answer:

When: API server became unreachable — all kubectl commands failing with TLS handshake errors.

Diagnosis: used crictl directly on control plane node (kubectl not working). crictl ps confirmed apiserver container running but rejecting connections.

kubeadm certs check-expiration: confirmed multiple certs expired.

Resolution: kubeadm certs renew all → restarted static pod manifests → updated admin.conf kubeconfig.

Verification: kubectl get nodes — cluster accessible. All services healthy.

Prevention: added Prometheus alert for cert expiry <30 days. Added Dynatrace synthetic monitor for API server health. Runbook created for future occurrences.

Result: cluster restored within SLA. Zero repeat incidents since.

Tip:

This is a strong differentiator — most candidates have not handled certificate expiry in production. Go into detail confidently.

Scenario

SSL certificate expired in production. Immediate steps?

Answer:

Don't panic — follow the runbook.

Confirm expiry: openssl s_client -connect :443 | openssl x509 -noout -dates — check actual cert expiry.

Raise emergency Change Request in ServiceNow — mark as emergency change.

Obtain renewed certificate from CA (or internal PKI team).

Import into keystore: keytool -importcert -alias -file new-cert.pem -keystore keystore.jks.

Restart application to load new cert (or hot reload if supported).

Verify: openssl s_client -connect :443 — check new expiry date.

Post-incident: update monitoring to alert 30 days before expiry. Update runbook.

Final Preparation Checklist

- Practice self-introduction out loud — time yourself. Short version: 45 seconds. Long version: 90 seconds.
- Be able to draw the on-prem traffic flow: DNS → VTM/HAProxy → K8s Service → Pod → DB/NFS
- Be able to draw the AWS traffic flow: Route 53 → IGW → ALB → EKS Node → Service → Pod → RDS/EFS
- Know the CI/CD pipeline steps by heart — from git commit to live pod — with tool at each step
- Know your key metrics: 60% deployment reduction · 40% build consistency · 30% incident resolution · 99.9% uptime · 20+ microservices · 110+ servers
- Practice the certificate expiry incident story — it is your strongest differentiator
- Know kubectl commands: get pods, logs --previous, describe, cordon, drain, uncordon
- Know Terraform: what remote state is, why DynamoDB, what state locking prevents
- Know ArgoCD: what OutOfSync means, how rollback works, difference between sync and self-heal
- Know AWS: public vs private subnet, IGW vs NAT Gateway, ALB vs NLB, IRSA vs access keys, Multi-AZ failover
- Be honest about depth — Helm used for community charts, not primary deployment method
- For any question: answer what you DID, tools you USED, result you ACHIEVED

Your resumes are strong. Your experience is real. Your metrics are your differentiators. Answer confidently, be specific, and never claim experience you don't have. Good luck, Narendra.